

Tool Specification Standard

Everything the standard requires, on the fewest pages that can carry it. Written to be kept open while you work — not to be read first.

Companion to Enterprise Tool Specification Standard, v1.0.0-rc1 · **Read that first.** Section references here (§7.3) point into it.
Owner Enterprise Architecture — Agent Platform · **Classification** Internal · 113 normative rules, all indexed in section 13.

USE THIS FOR

- Writing a descriptor with the rules in front of you
- Reviewing someone else's in twenty minutes
- Settling an argument by rule number
- Remembering which of the fifteen error codes applies

DO NOT USE THIS FOR

- Learning the model — the reasoning lives in the book
- Deciding tool boundaries — that is §12, and it needs the prose
- Quoting to a regulator — cite the standard, not the card

Conformance levels

LEVEL	NAME	REQUIRES	PERMITTED EXPOSURE
L0	Registered	Parses against the descriptor schema.	Dev and test only.
L1	Conformant	Every MUST satisfied; linter clean; named owner; structured errors; audit events.	Internal agents, non-customer data, tier 0–1.
L2	Certified	L1 + security review + evaluation evidence + entitlement sign-off + SLO + kill switch + retention.	Mandatory for customer data, money, or positions.
L3	Externally exposed	L2 + egress assessment + contractual controls + tenant isolation proof + annual red team.	Agents the firm does not operate.

Reading a rule

MUST	Gating. Fails the build and the review.
SHOULD	Departure needs a recorded reason.
MAY	Permitted, no expectation.
NEVER	MUST NOT, with no exception path.

TS-INP-09

```

┌───┐
│   │└─ number, permanent, never reused
│   │  area: GEN NAM DSC INP OUT ERR
│   │  ANN GOV SEC OBS VER CAT
└───┘  the standard

```

```
$ tools/lint-descriptors.py catalog/
FAIL brokerage_x TS-INP-09 ...
```

1 The descriptor

Eight top-level fields. No others — extensions go inside `_meta` under a namespaced key (`TS-GEN-02`).

```

{
  "name":      "brokerage_positions_list",      // MUST  ^[a-z][a-z0-9_]{2,63}$ · permanent · §5
  "title":     "List the holdings in an account", // MUST  ≤60 chars · for humans approving/auditing · §5.4
  "description": "Returns the current open holdings ...", // MUST  7 parts · ≤2,000 chars · §6

  "inputSchema": {                               // MUST  closed object schema · §7
    "type": "object",
    "properties": { /* every one described */ },
    "required": ["accountId"],                    //      explicit, even when empty
    "additionalProperties": false                  //      always false
  },
}
```

```

"outputSchema": { /* ... */ },                // MUST required by this standard · §8

"annotations": {                             // MUST all four, explicitly · §9
  "readOnlyHint": true,                       // changes nothing, anywhere
  "destructiveHint": false,                   // may be irreversible
  "idempotentHint": true,                     // repeat call has no extra effect
  "openWorldHint": false                     // reaches outside the firm
},

"icons": [ /* ... */ ],                       // MAY decorative only, never semantic

"_meta": {                                    // MUST never enters the model's context · §10
  "com.example.tooling/governance": { /* version, lifecycle, riskTier, owner,
    conformanceLevel, dataClassification, containsPii, piiFields, entitlements,
    approval, recordKeeping, cost, rateLimit, dependencies, availability,
    evaluation, deprecation */ }
}
}

```

THE RATIO TEST

About half the document is prose addressed to a model and a quarter is metadata addressed to a governance process. If yours inverts that, it is an API endpoint wearing a costume.

2 Naming & the operation vocabulary

```

// wire name – invocation, context, audit
brokerage_positions_list
├── brokerage
│   └── positions
│       └── list
└── operation, from the
    └── closed vocabulary
        ├── entity: singular = one,
        │   plural = many
        └── business domain, never a
            team, system, or vendor

// catalog id – registry, policy, docs only
brokerage.positions.list
// dots NEVER cross the wire: several runtimes
// reject or mangle them.

```

Grammar	<code>^[a-z][a-z0-9_]{2,63}\$</code> , 2–4 segments
Target length	≤ 40 chars SHOULD
Uniqueness	Firm-wide, case-insensitive, underscores ignored
Stability	Permanent. A rename is a new tool plus deprecation
Never contains	Environment, region, tenant, vendor, version (except a deliberate <code>_v2</code> successor)

REJECTED FINAL SEGMENTS

do run execute process handle manage perform fetch
query call , and any noun used as a verb.

The closed operation vocabulary

VERB	MEANS	ANNOTATIONS	RETURNS
<code>get</code>	Retrieve one identified entity	<code>read-only</code> <code>idempotent</code>	One object, or <code>NOT_FOUND</code>
<code>list</code>	Bounded, filterable collection	<code>read-only</code> <code>idempotent</code>	Array + <code>completeness</code> . Empty is success
<code>search</code>	Rank by relevance when the filter cannot be structural	<code>read-only</code> <code>idempotent</code>	Ranked array with scores
<code>summarize</code>	Derived aggregate instead of the records	<code>read-only</code> <code>idempotent</code>	One aggregate, basis stated
<code>check</code>	Evaluate a condition — the "can I?" tool	<code>read-only</code> <code>idempotent</code>	Verdict + reasons + the numbers
<code>preview</code>	Compute the effect without applying it	<code>read-only</code> <code>idempotent</code>	Projected outcome + <code>confirmationToken</code>
<code>create</code> , <code>place</code>	Bring a new entity into existence	<code>writes</code> <code>needs key</code>	The entity and its identifier
<code>update</code> , <code>replace</code>	Modify an existing entity	<code>destructive</code> <code>needs key</code>	The updated entity and what changed
<code>cancel</code>	Stop something in flight; the record survives	<code>destructive</code> <code>idempotent</code>	The entity's new state
<code>delete</code>	Remove an entity — rare and always suspicious	<code>destructive</code> <code>idempotent</code>	Confirmation and what was removed

VERB	MEANS	ANNOTATIONS	RETURNS
export	Produce a document for a human	read-onlynot idempotent	A reference, media type, size — never bytes

WHY THE VOCABULARY IS CLOSED

It lets the linter check annotations against the name, lets policy be written against a name rather than an argument, and lets a reviewer infer a tool's shape before reading a word of prose. Extensions go to the ARB and are granted for new *shapes*, never for synonyms.

Titles

YES	NO
"Get order status"	"OMS order status retrieval (v2)"
"List the holdings in an account"	"Order cancellation"
"Place a real order — trades money"	"View order" on a tool that cancels
Sentence case · ≤60 chars · no trailing period · tier 3 names the consequence, not the operation.	A title that misstates the operation is a control failure, not a typo.

3 The description

The model sees **only** name, title, description, and the input schema — including every property description. Nothing else. It reads them on every turn.

The seven parts

#	PART	WRITE	TEST
1	Purpose MUST	One sentence, active voice, starts with the verb.	Could a domain reader say what comes back?
2	Use when MUST	The questions this answers, phrased the way a customer phrases them.	At least two real user phrasings?
3	Do not use MUST	Nearest neighbors, by exact wire name , with the distinction.	Is every tool a confused reader might pick named here?
4	Argument semantics	Provenance of identifiers, units, cross-field rules.	Could a careful caller still supply a well-formed but wrong value?
5	Result orientation MUST	Shape in a sentence, domain conventions, what empty means .	Can the reader compute the right answer from the payload alone?
6	Effects, freshness, cost MUST	State change, money, staleness, rate limits.	Would a compliance reviewer be surprised by an omission?
7	Failure guidance	The failures a caller can act on, and what to do.	For each code: retry, change an argument, or stop?

BUDGETS

ITEM	TARGET	HARD
Description	80–180 words	2,000 chars
Per-parameter	1–2 sentences	240 chars
Whole descriptor	≤ 700 tokens	1,200 tokens
Tools per agent	≤ 25	40
Catalog share of context	≤ 10%	15%

CUT FIRST / KEEP ALWAYS

"This tool allows you to..." · restating the schema · which service or vendor · exhaustive error lists · how to talk to the customer · legal disclaimers
Every convention that makes a number mean something other than its face value · every exclusion naming a sibling · what an empty result means · any staleness the customer would be misled by

DESCRIPTIONS ARE ATTACK SURFACE

A description is an instruction injected into an agent's context. It is static, reviewed, signed, and hash-pinned — never generated at runtime, never templated with request data (TS-DSC-07 , TS-SEC-09). It never instructs the agent about anything but itself (TS-DSC-06 , TS-DSC-10).

4 Input schema

PERMITTED KEYWORDS

```
type properties required description enum const default
examples minimum maximum exclusiveMinimum exclusiveMaximum
multipleOf minLength maxLength pattern items minItems
maxItems uniqueItems $ref $defs oneOf (discriminated, ≤3
branches)
format : only date date-time duration uuid uri
```

PROHIBITED CONSTRUCTS

```
if / then / else dependentSchemas dependentRequired — model
accuracy collapses on conditional structure
not anyOf allOf — unreadable as instruction
patternProperties propertyNames additionalProperties: true —
unvalidatable open maps
$dynamicRef , recursive $ref — unbounded
nullable — not a JSON Schema keyword; use ["string", "null"]
Remote $ref at invocation time
```

Shape rules

Root	<code>{"type": "object"}</code> + explicit <code>properties</code> + explicit <code>required</code> + <code>"additionalProperties": false</code> . A no-argument tool still declares all four.
Size	≤ 12 top-level properties, ≤ 2 levels deep SHOULD . Beyond that the tool is doing two jobs.
Names	<code>lowerCamelCase</code> , domain vocabulary, and the same name means the same thing in every tool in the catalog.
Booleans	Named positively, defaulting to the conservative value: <code>includeClosed: false</code> , never <code>excludeClosed: false</code> .
Identifiers	Opaque, prefixed, pattern-constrained — <code>acct_</code> , <code>ord_</code> , <code>inst_</code> .
Bounds	Every array declares <code>maxItems</code> ; every free-text string declares <code>maxLength</code> .
Required	Only if the tool cannot proceed without it. Marking a field required to make the model "think about it" makes it invent a value.
Defaults	The narrow, least-surprising choice — and stated in prose , not only in <code>default</code> .

Make illegal states unrepresentable

```
"status": { "type": "string" },
"symbol": { "type": "string" },
"quantity": { "type": "number" },
"since": { "type": "string" },
"limit": { "type": "integer" }

// "open" "OPEN" "working" "aapl"
// "AAPL" "100.5" "last week" "10000"
```

```
"status": { "enum": ["WORKING", "FILLED", ...] },
"symbol": { "pattern": "^[A-Z]{1,5}$" },
"quantity": { "type": "integer",
               "minimum": 1, "maximum": 100000 },
"fromDate": { "format": "date" },
"pageSize": { "minimum": 1, "maximum": 100 }

// one way to be right
```

Enumerated values are `SCREAMING_SNAKE_CASE` and match Appendix D of the standard exactly. Where an enumeration is long or grows often, use a `pattern` plus a prose pointer to the tool that enumerates valid values. Where it is short and consequential — status, side, time in force, account type — spell it out. When two shapes genuinely differ, the answer is **two tools**, not a union.

Reserved parameter names

PARAMETER	SHAPE	SEMANTICS
<code>pageSize</code>	integer 1–100, default 25	Never <code>limit</code> , <code>count</code> , or <code>top</code> .
<code>cursor</code>	string ≤512	Opaque forward token. Offset and page numbers are prohibited against a moving book.
<code>asOf</code>	date-time	Point-in-time read; omitted means now. A tool that cannot honor history must not accept it.
<code>fromDate</code> , <code>toDate</code>	date	Inclusive window in the account's reporting time zone; declare the max span.
<code>detail</code>	<code>SUMMARY</code> <code>STANDARD</code> <code>FULL</code>	Verbosity. Preferred over a caller-supplied field list.
<code>idempotencyKey</code>	uuid	Required on every non-idempotent state change. Dedupe ≥24h; replay returns the original result.
<code>confirmationToken</code>	string ≤512	From the matching <code>preview</code> . Single use, ≤120 s, bound to the arguments.
<code>reason</code>	string ≤280	Why the action is being taken. Captured verbatim into the audit record. Required at tier 3.

Prohibited input patterns

NEVER ACCEPT	BECAUSE
Identity <code>userId</code> <code>customerId</code> <code>onBehalfOf</code> <code>role</code> <code>tenantId</code>	Access control delegated to a language model. It will be talked into acting as someone else.
Credentials <code>apiKey</code> <code>token</code> <code>password</code>	Secrets must never enter a context window — logged, retained, possibly forwarded.
Query languages <code>sql</code> <code>filterExpression</code> <code>graphqlQuery</code> <code>jsonPath</code>	Unvalidatable, unauditable, injection surface with the model as injector.
Discriminators <code>action</code> <code>operation</code> <code>mode</code> <code>command</code>	Collapses several risk profiles into one descriptor. Annotations, tier, approval, and audit all become wrong at once.
Safety flags <code>force</code> <code>skipValidation</code> <code>dryRun</code> <code>override</code>	A flag that changes whether a tool is dangerous means the annotations describe one of two tools. Split them.
Caller-supplied URLs or paths	Server-side request forgery primitive. Allow-list in configuration, never in an argument.
Unbounded arrays and blobs	Cost, latency, and denial-of-service vector.
Routing hints <code>env</code> <code>region</code> <code>useCache</code> <code>debug</code>	Deployment facts the model must not be able to reason about or influence.
Anything the tool cannot honor	The model will use it and silently receive an answer that does not match the request.

5 Output & the result envelope

```
{
  "content": [{ "type": "text", "text": "3 positions in ....8820 as of 14:32:05Z ..." }],
                // MUST ≥1 text block · generated FROM structuredContent, never hand-kept
                // MUST carry as-of, currency, sign, and any truncation · keep it compact
  "structuredContent": { // MUST validate against outputSchema · the authoritative payload
    "asOf": "2026-09-19T14:32:05Z", // MUST on anything that can change
    "basis": "DELAYED_15M", // MUST on any computed or convention-dependent figure
    "completeness": { "returned": 25, "totalMatching": 340,
                      "truncated": true, // MUST on every collection; never null
                      "nextCursor": "eyJvIjoyNX0",
                      "narrowingHint": "Filter by symbol rather than paging." },
    "disclosures": ["Prices are delayed up to 15 minutes ..."] // compliance text lives here,
  }, // not in the description
  "isError": false // true = the tool ran and could not do its job. Transport failures never reach it.
}
```

RESULT BUDGETS

RESULT	TARGET	HARD
Single entity	< 500 tok	2,000
Collection page	< 1,500 tok	5,000
Aggregate	< 800 tok	2,000
Error	< 200 tok	500
Binary / >100 KB	By reference only	

CUT FROM EVERY PAYLOAD

Nulls · internal identifiers · hypermedia links · derivations the model can safely redo · per-row repetition of `accountId` , `currency` , `asOf` · pre-formatted display strings.

NUMBERS

Money, quantities	Decimal strings — <code>"62137.50"</code> . Never IEEE-754.
Currency	ISO 4217, at the value or the envelope.
Formatting	None in <code>structuredContent</code> . No separators, symbols, or parenthesized negatives.
Timestamps	RFC 3339 with explicit UTC offset. Dates that are dates name their time zone.
Signs	Explicit in the text rendering: <code>-98,240.00 USD</code> , <code>SHORT 400</code> .

Empty, partial, stale, truncated

SITUATION	REQUIRED BEHAVIOR
Empty	<code>isError: false</code> , empty array, and prose saying what the emptiness means. A model shown a bare <code>[]</code> reports an outage about half the time.

SITUATION	REQUIRED BEHAVIOR
Not found	<code>NOT_FOUND</code> for a <code>get</code> ; an empty collection for a <code>list</code> . "That order does not exist" and "you have no orders" are different customer answers.
Truncated	<code>truncated: true</code> and words in the text: "showing 25 of 340". Tell the reader how to narrow , not how to page — a model follows a cursor once, maybe twice, then summarizes what it has as if it were everything.
Partial	Only where the missing part is separable and disclosed. Name what is missing, in the text.
Partial, and the number gates a decision	Fail instead. Balances, buying power, anything a trade depends on, must never be returned partially computed (<code>TS-OUT-11</code>).
Stale	Return it with an honest <code>asOf</code> and <code>basis</code> , plus <code>stale: true</code> past the freshness target. Let the reader decide; do not decide by hiding it.

6 Annotations, risk tiers, approval

ANNOTATIONS ARE ADVISORY — NEVER A CONTROL

An annotation is an assertion by whoever wrote the descriptor. For a third-party tool it is self-certification. **No security control may depend on one.** Controls derive from the **risk tier**, which the firm sets, reviews, records, and signs. Where the two disagree, the linter fails the build and a human decides which was wrong.

THE FOUR HINTS — DEFAULTS ARE PESSIMISTIC

ANNOTATION	DEFAULT	TRUE MEANS
<code>readOnlyHint</code>	<code>false</code>	Changes nothing anywhere — caches, counters, search history included.
<code>destructiveHint</code>	<code>true</code>	May be irreversible or overwrite state. Only meaningful when not read-only.
<code>idempotentHint</code>	<code>false</code>	Repeating the identical call has no additional effect. This is what tells a runtime a timeout is safe to retry.
<code>openWorldHint</code>	<code>true</code>	Reaches outside the firm's controlled domain.

All four stated explicitly, even at their defaults — an omission means the reviewer cannot tell whether the author decided or forgot.

QUICK CLASSIFICATION

Reads only, closed domain	<code>T0</code> / <code>T1</code>
Touches customer or position data	<code>T1</code>
Reversible, non-financial write	<code>T2</code>
Moves money or creates exposure	<code>T3</code>
Cancels a live order	<code>T3</code>
Disputed at review	the higher tier

CANCELLATION IS TIER 3

It feels like undoing something. It is not: an order that does not fill in a moving market is a realized economic difference, and you cannot un-cancel.

Risk tiers and their baseline controls

TIER	SCOPE & BROKERAGE EXAMPLES	APPROVAL	BASELINE CONTROLS
<code>T0</code> Reference	Public or firm-wide reference. No customer, position, or money. Market calendar · symbol lookup · contract specs · fee schedule	<code>none</code>	Audit event. Authentication only. Freely available to autonomous agents.
<code>T1</code> Customer read	Reads customer-identifying or position data. Changes nothing. Accounts · positions · balances · order status · transactions	<code>none</code>	Entitlements at all three points. Full argument-and-result audit. Per-principal rate limit. L2 conformance.
<code>T2</code> Reversible change	Changes state, reversibly, not financially material. Nickname · price alert · statement request · watchlist	<code>notify</code>	T1 + idempotency key + explicit notification + a documented reversal path.

TIER	SCOPE & BROKERAGE EXAMPLES	APPROVAL	BASELINE CONTROLS
T3 Material	Moves money, creates exposure, or is irreversible. Place, replace, cancel an order · transfer funds · change beneficiaries	<code>confirm</code>	T2 + confirmation bound to an argument hash + two-step preview/execute + <code>reason</code> captured + dedicated kill switch + heightened monitoring + independent security review.

Approval modes

MODE	DEFAULT FOR	BEHAVIOR
<code>none</code>	T0, T1	Invoked without interruption. Audited.
<code>notify</code>	T2	Executed immediately; the principal is told, with the reversal path.
<code>confirm</code>	T3	The principal sees exactly what will happen and approves before execution.
<code>dualControl</code>	exception	Two humans, neither the requesting agent's principal.
<code>humanExecution</code>	exception	The agent may only prepare; a human executes elsewhere.

What a confirmation must show

A RUBBER STAMP

Allow `brokerage_order_place?`

```
{ "accountId": "acct_01H08Z4M2X9K3P7R",  
  "symbol": "AAPL", "side": "BUY",  
  "quantity": 100, "orderType": "MARKET" }
```

[Allow] [Deny]

AN APPROVAL

Place a real order – this trades money

Buy 100 shares of AAPL (Apple Inc.)
Account ••••4471 · Roth IRA
Market order, good for today
Estimated cost 24,875.00 USD
Buying power after 59,485.20 USD

Fills immediately and cannot be undone.

[Place order] [Cancel]

- Rendered from the descriptor's `confirmationTemplate` with validated arguments — the action, the account in customer terms, the instrument in words, quantity and money, reversibility, and the material consequence.
- **Bound to a hash of the exact arguments shown.** If anything differs at execution, refuse.
- Expires — 120 s default — and is single use.
- A tool can never lower its own approval requirement through an argument, a description, or a returned instruction.

7 Governance block

Lives in `_meta` under a namespaced key, so it never competes for the model's context. Required members in **bold**.

version	Semantic. §17.2	approval	<code>{mode, confirmationTemplate, expiresInSeconds}</code>
lifecycle	<code>draft</code> <code>active</code> <code>deprecated</code> <code>retired</code>	recordKeeping	<code>{auditEvent, retention, regulatoryBasis}</code> . ISO 8601 duration
conformanceLevel	<code>L0</code> – <code>L3</code>	rateLimit	<code>perPrincipalPerMinute</code> at minimum
riskTier	<code>0</code> – <code>3</code>	cost	<code>{class, p95LatencyMs, unitCost}</code>
owner	<code>{team, contact, escalation}</code> . A distribution list is not an owner	dependencies	Systems of record, by business-capability name
dataClassification	Highest the tool can return	availability	<code>{slo, killSwitch, degradedMode}</code>
containsPii , piiFields	JSON Pointers, for mechanical masking	evaluation	Required at L2. <code>{suite, lastRunAt, selectionAccuracy, refusalAccu</code>
entitlements	<code>domain:resource:action</code> . Empty is a positive assertion, scrutinized at review	deprecation	<code>{since, sunset, successor, migrationNote}</code>

CLASSIFICATION

public	No restriction
internal	Employee agents only
customer-confidential	Accounts, positions, balances, orders. Entitlement-gated, excluded from training and vendor retention
restricted	Must not be returned by a tool. Masked to last four; unmasked needs a separate T3 dual-control tool

LATENCY CLASS

low	< 500 ms	Freely composable
medium	< 2 s	Avoid fan-out
high	< 10 s	Warn in the description
async	> 10 s	Must use the job pattern

ENTITLEMENTS ARE ENFORCED THREE TIMES

(a) catalog visibility — an agent is never offered a tool its principal cannot use; (b) the gateway, before invocation; (c) the implementation, against the specific resource in the arguments. (c) is the one that stops cross-customer access and the one most often skipped.

8 Errors

Domain and business failures are **execution errors**: a successful transport response with `isError: true`. Never raised as transport or protocol errors — those get retried blindly, are often invisible to the model, and reach the customer as "something went wrong".

```
{ "content": [{ "type": "text", "text": "Cannot place this order: the account has 84,360.20 USD of stock buying power and the order requires 123,750.00 USD. Reduce the quantity to 68 shares or fewer, or choose a different account." }],
  "structuredContent": { "error": {
    "code": "BUSINESS_RULE_VIOLATION", // MUST closed taxonomy · the only field safe to branch on
    "domainCode": "INSUFFICIENT_BUYING_POWER", // SHOULD registered · what analytics group by
    "message": "Order requires 123,750.00 USD ...", // MUST 1–2 sentences, with the numbers
    "retryable": false, // MUST explicit boolean, never inferred from prose
    "retryAfterSeconds": null, // when retryable
    "remediation": "Reduce quantity to 68 shares or fewer ... Do not retry this order unchanged.",
    "details": { "required": {...}, "available": {...}, "maxQuantity": 68 },
    "traceId": "01HQ9D5P8R2T6V9Y0B3D5F7H" // MUST the only internal identifier allowed out
  } },
  "isError": true }
```

The fifteen codes

CODE	RETRY	MEANS & WHAT THE CALLER DOES
INVALID_ARGUMENT	No	Malformed or schema-violating. Fix the argument ; <code>details.violations</code> names it by JSON Pointer.
NOT_FOUND	No	Absent or outside the grant — deliberately indistinguishable, so the tool cannot enumerate identifiers. List first; do not guess another id.
AUTHENTICATION_REQUIRED	After re-auth	Absent, expired, or revoked credentials. The agent cannot fix this.
ENTITLEMENT_DENIED	No	Authenticated, not permitted. Ask the customer to grant access; never retry.
APPROVAL_REQUIRED	After approval	Present the confirmation, re-invoke with the token.
APPROVAL_DENIED	No	A human declined. Stop . Rephrasing and retrying turns one declined trade into three.
PRECONDITION_FAILED	Sometimes	Stale confirmation token, missing idempotency key, version mismatch. <code>details</code> names it.
CONFLICT	No	State does not permit it — already filled, account closed. Re-read the entity first.
BUSINESS_RULE_VIOLATION	No	Well-formed, permitted, and the domain said no. The request was fine; the world declined. Explain, do not retry.
RATE_LIMITED	After the wait	Honor <code>retryAfterSeconds</code> . Recurrence means the agent is looping.
TIMEOUT	If idempotent	Retry only with an idempotency key or a true <code>idempotentHint</code> ; otherwise re-read state to find out whether it happened.
UPSTREAM_UNAVAILABLE	With backoff	<code>details.system</code> names which, so the agent can say what is unavailable rather than apologizing vaguely.
DATA_INCOMPLETE	With backoff	Computable only partially, and the tool refused to return a misleading figure. Distinct from upstream failure: the tool <i>did</i> reach its sources.

CODE	RETRY	MEANS & WHAT THE CALLER DOES
NOT_SUPPORTED	No	Well-formed request for something this tool does not do. <code>remediation</code> names the tool that does.
INTERNAL_ERROR	Once	<code>traceId</code> and nothing else of substance. Common occurrence means errors are being mapped lazily.

THE DISTINCTION THAT MATTERS MOST

`INVALID_ARGUMENT` = **the caller is wrong**; change the argument and it works. `BUSINESS_RULE_VIOLATION` = **the caller is fine and reality declined** — insufficient buying power, market closed, instrument halted. A tool that conflates them makes every agent built on it worse: it retries a doomed order, then tells the customer "invalid request" when the honest answer was "the market is closed".

REMEDIATION RULES

- An **instruction in the imperative**, naming the argument to change or the tool to call instead. Apologies are not remediation.
- State the permissible bound — the max quantity, the valid window, the required level. "Too large" without "how large" forces a guessing loop.
- Where a retry is futile, say **"do not retry"** explicitly. Models are biased toward retrying; `retryable: false` alone is markedly weaker.
- Enumerate **every** validation violation, not just the first.

NEVER IN AN ERROR PAYLOAD

- Stack traces, exception classes, line numbers → `INTERNAL_ERROR` + `traceId`
- Raw upstream text, vendor codes, SQL → a mapped taxonomy code
- Hostnames, queue names, topology → `details.system` as a business name
- Full account numbers, government ids, dates of birth → the masked form
- Any data about another principal → nothing; return `NOT_FOUND`
- Whether an id exists but is not permitted → `NOT_FOUND`, uniformly

Mapping upstream failures

UPSTREAM	MAPS TO	NOTE
400	INVALID_ARGUMENT	If the gateway validated and upstream still says 400, the schema has drifted — alert.
401 / 403	AUTHENTICATION_REQUIRED / ENTITLEMENT_DENIED	Refresh once, then surface / never retried.
404	NOT_FOUND	Uniform for "absent" and "not yours".
409 / 412 , 428	CONFLICT / PRECONDITION_FAILED	Re-read before acting / name the missing precondition.
422	BUSINESS_RULE_VIOLATION	The mapping most often collapsed into <code>INVALID_ARGUMENT</code> .
429	RATE_LIMITED	Propagate <code>Retry-After</code> .
5xx , timeouts	UPSTREAM_UNAVAILABLE / TIMEOUT	Distinguish them: a timeout on a non-idempotent write means the outcome is unknown .
Unmapped	INTERNAL_ERROR + alert	An unmapped upstream failure is a contract change nobody was told about.

9 Decisions you will make repeatedly

One tool or two?

IF...	THEN
Stating the user question takes two sentences with an "or" in the middle	Two tools
One argument value makes it destructive	Two tools — annotations, tier, approval, and audit cannot all be right at once
You are reaching for <code>oneOf</code> with more than three branches	Two tools
The two candidates differ only in cardinality (one vs many)	One tool — a <code>list</code> that returns everything a <code>get</code> would
A common question needs ≥3 calls plus a domain convention the model must know	One composite — move the knowledge into the tool where it can be tested
You cannot write the one sentence that distinguishes two tools	Merge or re-scope, and put that sentence in both descriptions

Does it need an idempotency key?

`create` · `place` · `update` · `replace` → **yes, required**, deduped ≥24 h, replay returns the original result, same key + different arguments → `CONFLICT` · `cancel` · `delete` → naturally idempotent. · Read tools → no. · Say it in the description too: models supply a key far more reliably when told what it is for than when it merely appears in the schema.

How long will it take?

DURATION	PATTERN
< 10 s	Synchronous. Declare the latency class.
10 s – minutes	Job pattern: <code>..._start</code> returns a <code>jobId</code> ; <code>..._status</code> reports progress. The start tool is not read-only even if the result is.
Minutes – hours	Job pattern + out-of-band notification. The agent must not poll in a loop; return <code>nextPollAfterSeconds</code> .
Any timeout	State explicitly whether the underlying operation may still complete. "May still have been placed" is a different instruction from "was not placed", and the agent cannot work it out.

Polling anything with a lifecycle

Return `isTerminal` and say in the description that a terminal entity must never be polled again — this one flag removes most wasteful polling. Return `nextPollAfterSeconds` scaled to the entity (2 s for a market order, 60 s for a transfer). Prefer telling the customer the firm will notify them over having an agent hold a conversation open as a timer.

10 Security, audit, versioning

The five security principles

The agent has no authority of its own	Every action runs as a principal, with that principal's scopes. There is no service account "the assistant" uses to read customer accounts — that account is a standing breach waiting for a prompt.
Least privilege, per invocation	Scopes declared per tool, checked per invocation, against the specific resource — not once per session.
Model output is untrusted input	Validate and authorize arguments as though they came from an anonymous internet client, because in the relevant sense they did.
Tool output is untrusted input to the model	Customer- and counterparty-supplied strings are data, never instruction.
Controls live where they cannot be talked out of	The gateway and the implementation. Not the description, not the annotations, not the system prompt — all three are inputs to a system that can be persuaded.

NON-NEGOTIABLES

- No argument asserts identity, role, or tenancy (`TS-SEC-02`)
- Authorize the **specific resource** in the arguments, every call (`TS-SEC-08`)
- Never forward an inbound token downstream — exchange it for one audiencced to that system (`TS-SEC-10`)
- No credentials in arguments, results, errors, or descriptions (`TS-SEC-03`)
- Confirmation tokens: bound to principal + tool + argument hash, ≤120 s, single use (`TS-SEC-01`)
- Kill switch for every T2+ tool, ≤60 s, no deployment (`TS-SEC-12`)
- Descriptor hashes pinned; a changed hash is refused, not tolerated (`TS-SEC-09`)
- Never ask for credentials or one-time codes through a tool's request for input (`TS-SEC-05`)

UNTRUSTED TEXT IN THIS DOMAIN

Account nickname · order memo and client order id · instrument name and description · transfer memo and beneficiary name · corporate action notes · upstream error text.

Length-cap it, strip control characters and markup, **delimit and label it as untrusted in the text rendering**, and never interpolate it into a sentence that reads as narration by the tool (`TS-SEC-11`).

ALERT ON THESE

- Identical invocation repeated 3x in a conversation
- Repeated `ENTITLEMENT_DENIED` or `NOT_FOUND` — the signature of enumeration
- **Approval denied, then retried with altered arguments** — the highest-signal event in the estate; block the conversation
- Descriptor hash mismatch at a runtime

The audit record — one per invocation, every path

`traceId` · `conversationId` · `turnId` · `principal` and how the identity was established · `agent` and version · `toolName` · `toolVersion` · `descriptorHash` · validated arguments with `piiFields` masked · `outcome` (success, or `code` + `domainCode`, or the policy decision) · `resultDigest` with size and record count · `approval` (mode, who, when, the argument hash they approved, or the decline) · `entitlements` presented and the decision · `latencyMs` · `upstreamCalls`.

THE DESCRIPTOR HASH IS NOT OPTIONAL

Without it you cannot reconstruct what the model was told when it made the call — the first question asked in any investigation. For tier 3, the record must also carry the confirmation exactly as the principal saw it and their response. "The agent placed an order" is a summary of a record, not a record.

METRICS WORTH THE INSTRUMENTATION

Selection accuracy	Was this the right tool for the question? Sampled and human-labeled. The most valuable metric in the estate
Refusal accuracy	Correctly left alone when it should not answer
Repeat-call rate	>15% = descriptor defect. Fix the prose, not the agent
Truncation rate	High = wrong granularity
Post-call correction	User corrects the agent right after this tool returns
Error rate by <code>code</code>	A rise in <code>BUSINESS_RULE_VIOLATION</code> is a market condition; a rise in <code>INVALID_ARGUMENT</code> is a descriptor defect
Approval decline rate	Near zero means the confirmation is being rubber-stamped
Silent wrongness	Succeeded, valid data, wrong answer. Only measurable by reading conversations. Budget for it

BREAKING VS COMPATIBLE

Optional input parameter with a safe default	ok
New output field	ok
New member of an output enum	ok
Relaxing a constraint	ok
Clarifying prose	encouraged
Optional → required	breaking
Removing or renaming any field	breaking
New member of an input enum	breaking in effect
Tightening a constraint	breaking
Changing a unit, convention, or sign	breaking & dangerous
Changing omission behavior	breaking
Raising the risk tier	announce it

A breaking change is a new tool name with a `_v2` suffix, running alongside the original for the full window. The model has no upgrade schedule.

DEPRECATION, IN ORDER

1. **Announce** — `lifecycle: deprecated` + `{since, sunset, successor, migrationNote}`. Minimum 90 days or one reporting cycle.
2. **Stop new grants** — no longer offered to audiences that do not already have it.
3. **Warn out of band** — a notice in the result's `_meta`. **Do not change the description:** it costs tokens every turn, and a model told a tool is deprecated behaves unpredictably rather than migrating.
4. **Measure** — track by audience, contact each remaining consumer.
5. **Retire** — `NOT_SUPPORTED` naming the successor for 90 days, then withdraw. **The name is never reused.**

EVALUATION THRESHOLDS

Selection accuracy	≥95% (T1), ≥98% (T3)
Refusal accuracy	≥95%, ≥99% (T3)
Argument accuracy	≥98%
Interpretation accuracy	≥98%; 100% on domain conventions
Every suite carries: nearest-neighbor confusions · convention traps (short position, option contracts, unsettled cash, retirement account, partial fill, rejection) · empty and degraded results · refusals · injection. Re-run on every change, including a change to nothing but the prose.	

11 The brokerage reference catalog

Generated from `catalog/*.json` at build time, so this card is the catalog rather than a description of it. Full treatment in Part IV of the standard.

TOOL	TIER	CONTRACT	ANSWERS
<code>brokerage_accounts_list</code> List the customer's brokerage accounts	T1 none	<code>read-only</code> <code>idempotent</code> <code>brokerage:accounts:read</code>	Returns every brokerage account the customer has granted access to, with its type, registration, status, base currency, and options approval level.
<code>brokerage_balances_get</code> Get cash, buying power, and margin for an account	T1 none	<code>read-only</code> <code>idempotent</code> <code>brokerage:balances:read</code>	Returns the money in one brokerage account: cash and settled cash, buying power broken out by what it can be used for, margin detail, and the account's total value.
<code>brokerage_order_get</code> Get the status of one order	T1 none	<code>read-only</code> <code>idempotent</code> <code>brokerage:orders:read</code>	Returns the current status, fill progress, and executions of one specific order.
<code>brokerage_order_preview</code> Preview an order before placing it	T1 none	<code>read-only</code> <code>idempotent</code> <code>open world</code> <code>brokerage:orders:write</code>	Computes exactly what would happen if an equity order were placed — estimated cost, commission, resulting buying power, and any warnings — without placing anything.
<code>brokerage_orders_list</code> Find orders on an account	T1 none	<code>read-only</code> <code>idempotent</code> <code>brokerage:orders:read</code>	Returns orders on one account, filtered by lifecycle state, symbol, or date, most recent first.
<code>brokerage_positions_list</code> List the holdings in an account	T1 none	<code>read-only</code> <code>idempotent</code> <code>brokerage:positions:read</code>	Returns the current open holdings in one brokerage account: instrument, quantity, average cost, last price, market value, and unrealized profit and loss for each.
<code>brokerage_order_place</code> Place a real order — trades money	T3 confirm	<code>writes</code> <code>destructive</code> <code>idempotent</code> <code>open world</code> <code>brokerage:orders:write</code>	Places a live equity order in the market using a confirmation token obtained from <code>brokerage_order_preview</code> .

Conventions the model must be told

Every row is a fact that turns a correct payload into an incorrect customer answer if the reader does not know it. Discharge each one in the description, in the payload, or by moving the work into the tool.

CONVENTION	WRONG ANSWER IF UNKNOWN	DISCHARGED BY
Quantity is signed; negative is short	"You own 400 Tesla" when the customer is short 400	Description + explicit <code>side</code> + SHORT in the text rendering
Option quantity is in contracts	Exposure understated by 100×	<code>sharesEquivalent</code> computed server-side
Settled cash ≠ total cash	"You have \$12,000 available" → good-faith violation	Separate fields + <code>buyingPower.applicable</code>
Four different buying-power figures	A retirement account told it has 2× margin buying power	<code>applicable</code> + <code>applicableReason</code> , computed by the tool
Retirement accounts cannot borrow	Margin advice given to an IRA	<code>registration</code> in the result; <code>margin: null</code>
An order can be partially filled and still working	"Your order filled" when 40% filled	<code>filledQuantity</code> / <code>remainingQuantity</code> / <code>isTerminal</code>
Status values may be added	An unknown status read as terminal — a live order reported dead	<code>isTerminal</code> boolean, computed by the tool
Marks may be 15 minutes delayed	A stale price quoted as live in a fast market	<code>asOf</code> + <code>basis</code> + a disclosure
The account identifier is not the account number	An internal identifier read aloud to a customer	Two fields, each described with its audience

CONFUSABLE PAIRS — PUT THE DISTINGUISHING SENTENCE IN BOTH DESCRIPTIONS

<code>positions_list</code> ↔ <code>orders_list</code>	An order becomes a position only when it fills. "Did my trade go through" is orders; "what do I own" is positions.
<code>balances_get</code> ↔ <code>positions_list</code>	Balances are cash and buying power; positions are holdings.
<code>balances_get</code> ↔ <code>transactions_list</code>	A balance is a state; a transaction is an event. "Why did my cash drop" is transactions.
<code>orders_list</code> ↔ <code>executions_list</code>	One order, many executions at different prices. "What price did I get" may need executions.
<code>order_check</code> ↔ <code>order_preview</code> ↔ <code>order_place</code>	"Can I?" / "What exactly would happen?" (and a token) / does it. Only the third trades.

ORDER LIFECYCLE — BRANCH ON THE BOOLEAN, NEVER THE STRING

<code>WORKING</code> · <code>ISTERMINAL: FALSE</code>	<code>TERMINAL</code> · <code>ISTERMINAL: TRUE</code>
<code>PENDING_NEW</code> <code>NEW</code>	<code>FILLED</code> <code>CANCELLED</code>
<code>PARTIALLY_FILLED</code>	<code>REJECTED</code> <code>EXPIRED</code>
<code>PENDING_CANCEL</code> <code>PENDING_REPLACE</code>	<code>REPLACED</code>
<code>SUSPENDED</code>	
Poll after <code>nextPollAfterSeconds</code> / never poll, never offer to cancel	
A STATUS YOU DO NOT RECOGNIZE BELONGS ON THE LEFT New lifecycle states get added. Treating an unknown status as terminal tells a customer their live order is dead.	

Two-step state change

```
brokerage_order_preview → { canPlace, estimate{estimatedCost, buyingPowerAfter, isEstimate},
                           blockers[{code, message, maxPermissibleQuantity}], warnings[],
                           confirmationToken: "cnf_...", expiresAt } // T1, read-only, call freely

confirmation rendered from confirmationTemplate · principal approves · arguments hashed

brokerage_order_place ← { ...order..., confirmationToken, idempotencyKey, reason }
                      → { orderId, status, isTerminal, statusExplanation, replayed }
                        PRECONDITION_FAILED if the token expired or the order differs
```

THE TIMEOUT REMEDIATION, VERBATIM — THREE SENTENCES, THREE PREVENTED INCIDENTS

"Do not tell the customer the order failed. Either retry `brokerage_order_place` with the **same** `idempotencyKey` , which will return the original result if the order was placed, or call `brokerage_orders_list` to check. Never place a second order with a new key."

12 Anti-patterns

ANTI-PATTERN	WHAT GOES WRONG	RULE
The kitchen-sink tool one tool, an <code>action</code> enum	Annotations, risk tier, approval policy, and the audit record all become wrong at once. The audit says <code>orders_manage</code> when what happened was a cancellation.	TS-NAM-06
The <code>dryRun</code> flag	One token in a model-generated object is the only thing between a simulation and a real trade.	TS-INP-13
The API mirror one tool per endpoint	Forty tools, none answering a question anyone asks; selection accuracy collapses and every answer needs four hops.	TS-CAT-01
The free-text filter	Unvalidatable, unauditable, injection surface. If the query space is genuinely open, the tool is a database connection in a costume.	TS-INP-17
The identity parameter	Access control delegated to a language model.	TS-SEC-02
Silent truncation	"Your portfolio is worth \$310,000" when 25 of 340 positions came back. Nothing errors; the number is simply wrong.	TS-OUT-07
Empty as error	An account with no positions reported as an outage. Support ticket, no logs, nothing to find.	\$8.6
The raw upstream payload	200,000 tokens, a production hostname, an ETL batch id, and six fields the customer must never see.	TS-GOV-02
The apologetic error	The agent retries a doomed call three times, then tells the customer nothing useful.	TS-ERR-02
Description as documentation	600 words restating the schema and omitting every convention that matters.	TS-DSC-01
Description as system prompt	"Always greet the customer warmly and suggest our premium tier." A tool reaching into a surface it cannot see.	TS-DSC-06
The unowned tool	Still returning customer data two years after the team was reorganized. Works perfectly; accountable to no one.	TS-GOV-01

ANTI-PATTERN	WHAT GOES WRONG	RULE
The rename	Six years of audit records silently change meaning. A record-keeping defect, not a refactor.	TS-NAM-11

THE PATTERN BEHIND THE PATTERNS

Twelve of the thirteen are the same mistake: **optimizing for the convenience of the team building the tool rather than the reliability of the caller consuming it**. Each shortcut moves work from a place where it can be tested to a place where it can only be hoped for.

13 Complete rule index

All 113 normative rules, generated from the standard's source so it cannot drift. The § column points into the standard; the linter reports findings by these identifiers. Rule numbers are permanent: a withdrawn rule is marked withdrawn and its number is never reused.

RULE	LEVEL	§	REQUIREMENT
TS-GEN · GENERAL AND DESCRIPTOR STRUCTURE			
TS-GEN-01	MUST	1.3	Every tool reachable from a production agent MUST hold at least Level 1. Every tool that reads customer data, moves money, changes an order, or writes to a system of record MUST hold Level 2.
TS-GEN-02	MUST	4.1	A descriptor MUST NOT carry top-level fields other than the eight above. Extensions go inside <code>_meta</code> under a namespaced key. A runtime that encounters an unknown top-level field MUST reject the descriptor rather than ignore it: silent tolerance is...
TS-GEN-03	MUST	4.3	The descriptor is the source of truth and MUST be version-controlled in the owning team's repository, published to the catalog by an automated pipeline, and never edited in the catalog's user interface.
TS-GEN-04	MUST	4.3	Descriptor, implementation, and tests MUST version and deploy together. A descriptor describing behavior that is not yet deployed is a defect of the same severity as a wrong answer.
TS-GEN-05	MUST	4.3	Descriptors MUST be generated or validated by the conformance linter in CI. A build that emits a non-conformant descriptor fails.
TS-GEN-06	SHOULD	4.3	Schemas SHOULD be derived from the implementation's types rather than hand-written alongside them, so that drift is impossible rather than merely discouraged. Prose fields are hand-written; shapes are generated.
TS-GEN-07	MUST	4.3	A descriptor MUST be self-contained: every <code>\$ref</code> resolves either within the document or to a schema the catalog publishes and pins by immutable version. No unresolvable references, no network-dependent resolution at invocation time.
TS-GEN-08	NEVER	4.3	A descriptor MUST NOT embed credentials, internal hostnames, connection strings, or system topology. It is read by a model and may be logged in full.
TS-NAM · NAMING AND IDENTITY			
TS-NAM-01	MUST	5.1	A tool name MUST match <code>^[a-z][a-z0-9_]{2,63}\$</code> — lowercase ASCII letters, digits, and underscore; starting with a letter; between 3 and 64 characters.
TS-NAM-02	MUST	5.1.1	Names MUST be globally unique across the firm's catalog, compared case-insensitively and with underscores ignored. <code>orders_list</code> and <code>ordersList</code> collide.
TS-NAM-03	SHOULD	5.1.1	Names SHOULD be 40 characters or fewer. Long names crowd every prompt and get truncated in audit and approval surfaces.
TS-NAM-04	MUST	5.1.1	The name MUST NOT encode a version (<code>_v2</code>) unless the tool is a deliberate breaking successor published under §17.3, in which case it MUST.
TS-NAM-05	NEVER	5.1.1	Names MUST NOT contain an environment, region, tenant, or vendor (<code>_uat</code> , <code>_emea</code> , <code>_fiserv</code>). Those are deployment facts; a model that can see them will eventually reason about them.
TS-NAM-06	MUST	5.3	The final segment MUST come from the vocabulary above. Proposals to extend it go to the Architecture Review Board and are granted for genuinely new shapes of operation, not for synonyms.
TS-NAM-07	MUST	5.4	Every descriptor MUST carry a title: sentence case, no trailing punctuation, 60 characters or fewer, understandable to someone who does not know the system — "Get order status", not "OMS order status retrieval (v2)".
TS-NAM-08	MUST	5.4	The title MUST describe the same operation as the name. A title that says "View" on a tool that cancels is a control failure, not a typo.
TS-NAM-09	SHOULD	5.4	Titles of state-changing tools SHOULD lead with the verb and name the consequence: "Cancel a working order", not "Order cancellation".
TS-NAM-10	MAY	5.4	Icons MAY be supplied for catalog and approval surfaces. Provide SVG with a light and a dark variant. Icons are decoration: no control, policy, or model behavior may depend on one.
TS-NAM-11	MUST	5.5	A published tool name is permanent. It MUST NOT be reused for a different capability, and a capability MUST NOT be renamed. A name change is modeled as a new tool plus deprecation of the old one under §17.4.
TS-DSC · DESCRIPTIONS			
TS-DSC-01	MUST	6.2	Every description MUST contain parts 1, 2, 3, 5, and 6 below. Parts 4 and 7 are where they apply — which, for anything in a financial domain, is nearly always.
TS-DSC-02	MUST	6.3	A tool description MUST NOT exceed 2,000 characters. A description at the limit is usually a tool that should have been two.

RULE	LEVEL	§	REQUIREMENT
TS-DSC-03	MUST	6.3	A parameter description MUST NOT exceed 240 characters. Anything longer belongs in the tool description, where it is read once rather than per parameter.
TS-DSC-04	MUST	6.5	Descriptions MUST be plain prose in the active voice, present tense, starting with a verb. No markdown headings, no bullet syntax, no code fences — runtimes render descriptions inconsistently and structure is lost or corrupted.
TS-DSC-05	MUST	6.5	Sibling tools referenced in exclusions MUST be named by their exact wire name. "Use the balances tool" is not actionable; <code>brokerage_balances_get</code> is.
TS-DSC-06	MUST	6.5	Descriptions MUST NOT contain instructions about how the agent should behave outside this tool call — how to address the customer, what to disclose, when to escalate, what to do in other conversations. A tool describes itself;
TS-DSC-07	NEVER	6.5	Descriptions MUST NOT be generated at runtime, templated with request data, or personalized per user. A description is a static, reviewed, signed artifact. Anything dynamic belongs in the result payload.
TS-DSC-08	SHOULD	6.5	Descriptions SHOULD use the domain's own vocabulary — the words that appear on the customer's statement and in the firm's procedures. Inventing a synonym for a term of art makes the tool harder to select and its audit trail harder to read.
TS-DSC-09	SHOULD	6.5	Where an enumerated value has consequences, the description SHOULD state them once — "RESTRICTED accounts can be read but not traded" — rather than leaving the model to infer meaning from a token.
TS-DSC-10	MUST	6.6	A description MUST NOT reference a tool outside its own domain namespace except to exclude it, and MUST NOT contain imperative instructions addressed to the agent about tools other than itself. Descriptions that do are rejected at publish time.
TS-INP · INPUT SCHEMAS			
TS-INP-01	MUST	7.1	<code>inputSchema</code> MUST be a JSON Schema 2020-12 document whose root is <code>{"type": "object"}</code> with an explicit properties map, an explicit required array (possibly empty), and <code>"additionalProperties": false</code> .
TS-INP-02	SHOULD	7.1.2	An input schema SHOULD declare no more than 12 top-level properties and SHOULD NOT nest more than two levels deep. Beyond that, argument-composition accuracy degrades sharply and the tool is usually doing two jobs.
TS-INP-03	MUST	7.2	Property names MUST be lowerCamelCase and MUST use the domain's own vocabulary — <code>accountId</code> , <code>timeInForce</code> , <code>limitPrice</code> — not invented synonyms.
TS-INP-04	MUST	7.2	A property name MUST mean the same thing in every tool in the catalog. <code>quantity</code> cannot be <code>shares</code> in one tool and <code>contracts</code> in another. Where a concept recurs, take it from the shared type library (Appendix B) rather than redefining it.
TS-INP-05	SHOULD	7.2	Boolean parameters SHOULD be named positively and default to the conservative value: <code>includeClosed</code> defaulting to <code>false</code> , never <code>excludeClosed</code> defaulting to <code>false</code> . Double negatives are reliably mis-set.
TS-INP-06	SHOULD	7.2	Inputs SHOULD be flat. Nesting that exists only to mirror an upstream request body is noise; flatten it and let the implementation rebuild the shape.
TS-INP-07	MUST	7.2	Identifiers MUST be opaque, prefixed, and pattern-constrained (<code>acct_</code> , <code>ord_</code> , <code>inst_</code>). The prefix tells the model which identifier goes in which slot, and the pattern stops a well-formed-looking hallucination before it reaches a system of record.
TS-INP-08	MUST	7.2	Every array parameter MUST declare <code>maxItems</code> , and every string parameter MUST declare <code>maxLength</code> .
TS-INP-09	MUST	7.3.1	Any parameter whose valid values are a known finite set MUST be an enum. Enumerated values MUST be <code>SCREAMING_SNAKE_CASE</code> and MUST match the domain enumerations in Appendix D exactly — the same tokens the systems of record and the customer statement...
TS-INP-10	MUST	7.4	A parameter is required only if the tool genuinely cannot proceed without it. Marking a parameter required to force the model to "think about it" causes the model to invent a value instead.
TS-INP-11	MUST	7.4	Every optional parameter's omission behavior MUST be stated in its description in prose, not only in default. Models read prose far more reliably than they read schema annotations.
TS-INP-12	MUST	7.4	Defaults MUST be the narrow, conservative, least-surprising choice: the smallest page, the current account only, the shortest date window, exclusions on. A default that widens scope is a defect even when it is convenient.
TS-INP-13	NEVER	7.4	A parameter that changes the tool's safety characteristics — whether a change is simulated or real, whether validation is skipped, whether a confirmation is required — MUST NOT exist. Those are separate tools with separate annotations (§9.1, §25.2).
TS-INP-14	MUST	7.5	Every property in an input schema MUST carry a description. A property description MUST NOT restate the type, and MUST state, where applicable: where the value comes from, its unit or convention, and what happens if it is omitted.
TS-INP-15	MUST	7.6	Where a tool needs one of the reserved parameter concepts, it MUST use the standard name and shape rather than inventing an equivalent.
TS-INP-16	MUST	7.6	A tool MUST NOT accept a parameter it cannot honor. Offering an <code>asOf</code> on a tool with no history, or a currency on a tool that only reports in the account's base currency, is worse than omitting it: the model will supply the parameter and silently...
TS-INP-17	MUST	7.7	An input schema MUST NOT contain any of the patterns below. Each is rejected at review and by the linter; the right-hand column is the reason, not a mitigation.

RULE	LEVEL	§	REQUIREMENT
TS-OUT · OUTPUT AND ENVELOPES			
TS-OUT-01	MUST	8.1	Every tool MUST declare an outputSchema and MUST return both content and structuredContent on every invocation, including error invocations. Runtimes differ in which one they surface to the model;
TS-OUT-02	MUST	8.2	The text rendering MUST be derived from structuredContent by the implementation — generated, not hand-maintained — so the two cannot disagree.
TS-OUT-03	MUST	8.2	The text rendering MUST NOT contain any fact absent from structuredContent, and MUST carry the facts a reader could be materially misled by omitting: the as-of time, the currency, the sign of a position, and any truncation.
TS-OUT-04	SHOULD	8.2	The text rendering SHOULD be compact — a few hundred characters for a typical result — and SHOULD NOT be a pretty-printed dump of the structured payload. Duplicating the whole payload as text doubles the token cost of every call for no gain.
TS-OUT-05	MUST	8.2	Monetary and quantity values in the text rendering MUST carry their currency and their sign explicitly. "-98,240.00 USD" and "SHORT 400" are unambiguous; "98240" is a defect waiting for a customer complaint.
TS-OUT-06	MUST	8.4	Collections MUST use opaque forward cursors. Offset and page-number pagination MUST NOT be used against a mutable collection.
TS-OUT-07	MUST	8.4	When a result is truncated, completeness.truncated MUST be true and the text rendering MUST say so in words: "showing 25 of 340 positions". Silent truncation causes the model to state a partial total as a complete one.
TS-OUT-08	SHOULD	8.4	A truncated result SHOULD tell the reader how to narrow rather than how to continue: "filter by assetClass or symbol to see fewer than 340". Narrowing produces a correct answer in one more call; paging produces fourteen calls and an abandoned loop.
TS-OUT-09	SHOULD	8.4	Where a caller is likely to want the whole collection only in order to compute an aggregate, the catalog SHOULD offer a summarize tool that computes it server-side.
TS-OUT-10	MUST	8.5	Binary content and any text payload over 100 KB MUST be returned by reference rather than inline. Reference URIs MUST be short-lived and MUST require the principal's own authority to dereference — a link that anyone holding the URI can fetch has...
TS-OUT-11	MUST	8.6	Balances, buying power, and anything that gates a trading decision MUST NOT be returned partially computed; the tool fails instead.
TS-OUT-12	MUST	13.2	Every tool MUST enforce a result size ceiling in code and MUST truncate with disclosure (TS-OUT-07) rather than returning an unbounded payload. "The customer has 340 positions" is not an excuse; it is the case the ceiling exists for.
TS-OUT-13	MUST	13.5	Monetary amounts and quantities MUST be decimal strings — "62137.50" — never IEEE-754 numbers. Binary floating point cannot represent a cent, and the error is silent, compounding, and eventually a reconciliation break.
TS-OUT-14	MUST	13.5	Every monetary value MUST be accompanied by an ISO 4217 currency, at the value or at the envelope level. Never assume USD.
TS-OUT-15	MUST	13.5	Structured values MUST NOT carry locale formatting — no thousands separators, no currency symbols, no parenthesized negatives. Formatting belongs in the text rendering, where (2,110.00) is spelled -2,110.00 USD because parentheses for negatives are...
TS-OUT-16	MUST	13.5	Timestamps MUST be RFC 3339 with an explicit UTC offset. Dates that are genuinely dates — settlement date, expiry — are YYYY-MM-DD, and the description MUST name the time zone they are interpreted in.
TS-ERR · ERRORS			
TS-ERR-01	MUST	11.1	Domain and business failures MUST be returned as execution errors inside the envelope, never raised as transport or protocol errors.
TS-ERR-02	MUST	11.4	remediation MUST be an instruction in the imperative, naming the argument to change or the tool to call instead. Apologies are not remediation.
TS-ERR-03	MUST	11.4	Where a constraint was violated, the error MUST state the permissible value or bound — the maximum quantity, the valid window, the required approval level. An error that says "too large" without saying how large is permitted forces a guessing loop.
TS-ERR-04	MUST	11.4	Where a retry is futile, remediation MUST say so explicitly — "do not retry". Models are biased toward retrying, and an explicit prohibition is markedly more effective than retryable: false alone.
TS-ERR-05	MUST	11.4	Validation failures MUST enumerate every violation, not just the first. One round trip per invalid field is a conversation the customer watches.
TS-ERR-06	MUST	11.6	Every upstream failure mode MUST have an explicit mapping with a test. A default branch that maps everything unrecognized to INTERNAL_ERROR is permitted and MUST raise an alert when it fires: an unmapped upstream failure is a contract change nobody...

RULE	LEVEL	§	REQUIREMENT
TS-ANN · ANNOTATIONS AND RISK TIERS			
TS-ANN-01	MUST	9.1	All four annotations MUST be stated explicitly, even where the value equals the default. The defaults are deliberately pessimistic so that an omission fails safe;
TS-ANN-02	MUST	9.1	Annotations MUST be consistent with the operation verb in the tool's name (§5.3). A tool named ..._list with readOnlyHint: false fails the linter, and a tool named ..._place with readOnlyHint: true fails it harder.
TS-ANN-03	MUST	9.3	Every descriptor MUST declare a riskTier. A tool whose tier is disputed at review defaults to the higher tier until the dispute is resolved in writing.
TS-ANN-04	MUST	9.4.1	Tier 3 tools MUST supply a confirmationTemplate that renders, in the principal's language: the action, the identified account in customer-facing terms, the instrument in words, the quantity and the money, the reversibility, and the material...
TS-ANN-05	MUST	9.4.1	Approval MUST be bound to the exact argument set that was displayed. The runtime MUST hash the validated arguments, present them, and refuse execution if anything differs.
TS-ANN-06	MUST	9.4.1	Approvals MUST expire — 120 seconds is the default for trading actions — and MUST NOT be reusable for a second invocation.
TS-ANN-07	NEVER	9.4.1	A tool MUST NOT be able to lower its own approval requirement through an argument, a description, or a returned instruction. Approval policy is resolved from the catalog, before the implementation is reached.
TS-GOV · GOVERNANCE METADATA			
TS-GOV-01	MUST	10.2	Every tool MUST name an owning team with a paging escalation path. The catalog MUST re-verify ownership quarterly and automatically move a tool whose owner cannot be confirmed to deprecated, then retired ninety days later.
TS-GOV-02	MUST	10.3	A tool MUST return the minimum data that answers its question. Returning an entire upstream record because it was convenient is a data-minimization failure, and in an agent context the data does not merely sit in a response object — it enters a...
TS-GOV-03	MUST	10.4	Entitlements MUST be enforced at three points: (a) catalog visibility — an agent is not offered a tool its principal cannot use; (b) the gateway, before invocation; (c) the implementation, against the specific resource named in the arguments.
TS-GOV-04	MUST	10.5	Every tool MUST declare a rate limit per principal. Where a tool is expensive in money or in upstream capacity, the description SHOULD say so plainly — a model told that a tool is slow and expensive will use it more sparingly, which is a cheaper...
TS-GOV-05	MUST	14.5	A tool MUST NOT block for longer than its declared timeout, and MUST return TIMEOUT with an explicit statement of whether the underlying operation may still complete.
TS-SEC · SECURITY AND TRUST			
TS-SEC-01	MUST	14.2	A confirmation token MUST be bound to the principal, the tool, and a hash of the previewed arguments; MUST expire within 120 seconds by default; and MUST be single-use. It is a capability, not a receipt.
TS-SEC-02	NEVER	15.2	A tool MUST NOT accept an argument that asserts identity, role, tenancy, or entitlement. Identity arrives out of band, from the identity plane. A tool with an onBehalfOf parameter has delegated its access control to a language model.
TS-SEC-03	NEVER	15.2	Credentials of any kind MUST NOT appear in arguments, results, errors, or descriptions. Anything in those four places will be logged, retained in conversation history, and may be transmitted to a model provider.
TS-SEC-04	MUST	14.3	A request for input MUST be expressed as a schema of primitive fields — string, number, boolean, or enum — with a clear prompt, and MUST be declinable. An agent that cannot obtain the value returns to the user; it does not substitute a default.
TS-SEC-05	NEVER	14.3	Input requests MUST NOT be used to collect credentials, card numbers, government identifiers, or one-time passcodes. Authentication happens in the identity plane, on a surface the customer can verify — never inside a tool's request for arguments,...
TS-SEC-06	SHOULD	14.3	Prefer disambiguating in the schema over asking at runtime. A tool that returns INVALID_ARGUMENT naming the three eligible accounts is usually a better experience than a modal dialog, and always a simpler control.
TS-SEC-07	MUST	14.4	Every tool that is not naturally idempotent MUST require an idempotencyKey, MUST deduplicate on it for at least 24 hours, and MUST return the original result — not an error — when a key is replayed with identical arguments.
TS-SEC-08	MUST	15.2	The implementation MUST authorize the principal against the specific resource named in the arguments, on every invocation. Holding brokerage:positions:read authorizes reading your positions, not a position identifier you were able to produce.
TS-SEC-09	MUST	15.7	Every descriptor's content hash MUST be recorded at approval. Runtimes MUST pin the approved hash and MUST refuse to present or invoke a tool whose descriptor has changed without a new approval — for internal tools too, but decisively for external...
TS-SEC-10	NEVER	15.2	A token presented to the gateway MUST NOT be forwarded verbatim to a downstream system. Exchange it for a token whose audience is that system, scoped to this operation.

RULE	LEVEL	§	REQUIREMENT
TS-SEC-11	MUST	15.3	Free-text content that did not originate in a system the firm controls MUST be neutralized before it is returned: length-capped, stripped of control characters and markup, and MUST be delimited and labeled as customer-supplied or externally-supplied...
TS-SEC-12	MUST	15.6	Every tier 2 and tier 3 tool MUST be disableable within 60 seconds, by capability, by audience, and globally, without a deployment.
TS-SEC-13	MUST	15.7	Third-party tools MUST be reviewed against this standard before admission, MUST be re-reviewed on every descriptor change, and MUST be assumed to have mis-stated their annotations — the firm assigns the risk tier independently, from observed...
TS-SEC-14	MUST	15.7	A third-party tool MUST NOT be presented to an agent in the same context as a firm tier 3 tool unless the combination has been explicitly assessed. The risk is not either tool; it is that one tool's output can steer the other's invocation.
TS-OBS · OBSERVABILITY AND AUDIT			
TS-OBS-01	MUST	16.1	Every invocation — successful, failed, denied, or refused at policy — MUST emit exactly one audit record containing the fields below, retained for the period named in the tool's recordKeeping.retention.
TS-OBS-02	MUST	16.1	For tier 3 tools the audit record MUST be sufficient to reconstruct the decision: the arguments, the confirmation exactly as the principal saw it, their response, the result, and the descriptor version that produced the prose the model read.
TS-OBS-03	MUST	27.4	Tier 3 tools and every L3 exposure MUST be red-teamed before release and at least annually thereafter, by a team that did not build them.
TS-VER · VERSIONING AND LIFECYCLE			
TS-VER-01	MUST	17.2	governance.version MUST be semantic. Patch for prose and non-behavioral fixes; minor for compatible additions; major only alongside a new tool name (TS-VER-03).
TS-VER-02	MUST	17.2	Output enumerations MUST be documented as open: the description states that new members may appear and how to treat one. Order status is the canonical case — an unrecognized status means working, never terminal, and getting that default backwards...
TS-VER-03	MUST	17.2	A breaking change MUST be published as a new tool name with a _v2 suffix, running alongside the original. The two coexist for the full deprecation window.
TS-VER-04	MUST	17.2	Any change to prose, schema, or behavior MUST re-run the tool's evaluation suite before publication (§27.3). A description edit is a behavior change: it is the input that determines whether the tool is selected at all.
TS-VER-05	MUST	17.4	The platform team MUST maintain a register of the constraints imposed by every runtime the firm publishes to, MUST enforce the intersection of those constraints in the linter, and MUST re-validate the entire catalog whenever a runtime's constraints...
TS-CAT · CATALOG AND REGISTRATION			
TS-CAT-01	SHOULD	12.1	One tool SHOULD correspond to one decision the agent has to make, expressed as one question a user would ask — not to one resource, one endpoint, or one database table.
TS-CAT-02	SHOULD	12.4	An agent SHOULD be offered no more than 25 tools, and MUST NOT be offered more than 40 without a documented exception.
TS-CAT-03	MUST	12.4	The catalog MUST support audience-scoped views, so that a customer-facing assistant, an adviser's copilot, and an operations agent each see only their own set. Audience scoping is an entitlement decision (§10.4), not a UI preference.
TS-CAT-04	SHOULD	12.4	Where an estate genuinely exceeds what one agent can hold, the platform SHOULD scope by task phase — a small core set always present, with a task-specific set attached when the conversation enters that task — rather than presenting everything and...
TS-CAT-05	MUST	26.3	An exception to a MUST rule MUST be recorded in the catalog against the tool, with the rule identifier, the rationale, the compensating control, the approver, and an expiry of no more than twelve months.
TS-CAT-06	MUST	28.1	Be the single system of record for descriptors, serving them by name and version, with content hashes, to every runtime in the firm.
TS-CAT-07	MUST	28.1	Accept descriptors only from an authenticated publishing pipeline, never from a user interface, and MUST reject any that fail the linter or the meta-schema (TS-GEN-03, TS-GEN-05).
TS-CAT-08	MUST	28.1	Enforce global name uniqueness at publish time and refuse to reuse a retired name (TS-NAM-02, TS-NAM-11).
TS-CAT-09	MUST	28.1	Serve audience-scoped views resolved against the principal's entitlements, so a tool a principal cannot use is never placed in an agent's context (TS-GOV-03).
TS-CAT-10	MUST	28.1	Retain every published version indefinitely. An audit record naming a tool version is meaningless if that version cannot be retrieved.

RULE	LEVEL	§	REQUIREMENT
TS-CAT-11	SHOULD	28.1	Expose ownership, risk tier, lifecycle, dependencies, invocation volume, and evaluation results as a browsable inventory — the artifact a regulator asks for when they ask what the firm's agents can do.

14 Review checklist

The working document for a review meeting. Items in **bold** are gating: a failure means the tool does not ship. Full version with commentary is Appendix E of the standard.

Identity & description

- ☐ **TS-NAM-01** Name matches the grammar; domain segment present; final segment in the vocabulary
- ☐ **TS-NAM-02** Globally unique; never used before
- ☐ **TS-NAM-07** Title ≤60 chars, human-readable, names the consequence at tier 3
- ☐ **TS-DSC-01** Purpose, "use when", "do not use", result orientation, effects
- ☐ **TS-DSC-05** Nearest neighbors named by exact wire name
- ☐ TS-DSC-02 ≤2,000 characters
- ☐ **TS-DSC-06** No instructions about agent behavior outside this tool
- ☐ **TS-DSC-07** Static, reviewed, not templated at runtime
- ☐ §18.3 Every domain convention that could produce a wrong answer is stated

Schemas

- ☐ **TS-INP-01** Closed object; explicit `required`; `additionalProperties: false`
- ☐ **TS-INP-09** Every finite-valued parameter is an enum from Appendix D
- ☐ **TS-INP-07** Identifiers prefixed and pattern-constrained
- ☐ **TS-INP-08** `maxItems` on arrays, `maxLength` on free text
- ☐ **TS-INP-14** Every parameter description says more than its type
- ☐ TS-INP-12 Defaults are the conservative choice
- ☐ **TS-INP-17** No identity, credential, query language, discriminator, or safety flag
- ☐ TS-INP-16 No parameter the tool cannot honor
- ☐ **TS-OUT-01** Output schema declared; both `content` and `structuredContent` on every path
- ☐ **TS-OUT-03** `asOf` and `basis` on every figure that can change
- ☐ **TS-OUT-07** `completeness` present; truncation stated in the text
- ☐ **TS-OUT-13** Money and quantities are decimal strings with a currency
- ☐ §8.6 Empty is success; partial is disclosed or refused

Errors

- ☐ **TS-ERR-01** Domain failures are execution errors, not transport errors
- ☐ **TS-ERR-02** Imperative remediation naming the next action
- ☐ **TS-ERR-03** Constraint violations state the permissible bound
- ☐ **TS-ERR-06** Every upstream failure mapped and tested; default branch alerts
- ☐ **§11.5** No stack traces, hostnames, upstream prose, or unmasked identifiers
- ☐ §11.3 `INVALID_ARGUMENT` and `BUSINESS_RULE_VIOLATION` not conflated

Classification & control

- ☐ **TS-ANN-01** All four annotations, consistent with the operation verb
- ☐ **TS-ANN-03** Risk tier declared and defensible; state-changing ⇒ tier 2+
- ☐ **TS-ANN-04** Tier 3 has a confirmation template a human can act on
- ☐ **TS-ANN-05** Approval bound to an argument hash, and expires
- ☐ **TS-SEC-07** Non-idempotent tools require a key; dedupe 24 h
- ☐ **TS-SEC-02** No identity accepted from the caller
- ☐ **TS-SEC-08** Resource-level authorization shown by a negative test
- ☐ **TS-SEC-10** No token passed through downstream
- ☐ **TS-SEC-11** Externally-sourced free text neutralized and labeled
- ☐ **TS-SEC-12** Kill switch for tier 2+, tested

Governance & evidence

- ☐ **TS-GOV-01** Named owning team with a paging escalation path
- ☐ **TS-GOV-02** Minimum necessary data; classification and PII mapped
- ☐ **TS-GOV-03** Entitlements at catalog, gateway, and implementation
- ☐ **TS-GOV-04** Per-principal rate limit declared
- ☐ **TS-0BS-01** Audit record on every path, including the descriptor hash
- ☐ **TS-VER-04** Evaluation suite exists and has run against this version
- ☐ §27.3 Selection, refusal, argument, interpretation thresholds met
- ☐ **TS-SEC-09** Descriptor hash registered with the catalog
- ☐ TS-CAT-05 Exceptions recorded with a compensating control and an expiry

Catalog fit

- ☐ TS-CAT-01 Answers one question a user actually asks
- ☐ TS-CAT-02 The agent's catalog stays within 25 tools
- ☐ §12.5 One sentence distinguishes it from its nearest neighbor — in both descriptions
- ☐ §12.3 No common question needs ≥3 calls plus a convention the model must know

SIX QUESTIONS TO ASK OUT LOUD

1. **"Read me the description."** Unreadable aloud, or ninety seconds long, or no sentence starting "Do not use" — not ready.
2. **"What is the nearest tool, and what sentence tells them apart?"** A pause is a confusability problem.
3. **"What does the customer see if this is wrong?"** A specific wrong sentence, not "an error".
4. **"Show me the negative test"** where the customer asks for someone else's account. Not the code — the test.
5. **"Who is paged at 2am, and what do they turn off?"**
6. **"What did the evaluation say about the cases where it should refuse?"** Selection accuracy is easy to score well on. Refusal accuracy is where the risk lives.

EA-STD-TOOL-001 · Tool Specification Standard — Quick Reference · v1.0.0-rc1

Companion to the full standard; where the two differ, the standard governs. Enterprise Architecture — Agent Platform. Generated from the same sources as the standard: the rule index from `pdf-src/`, the catalog card from `catalog/`.